

AMANE — FRAMEWORK SAAS SOUVERAIN

Note de cadrage technique

Choix de langages (Rust / Go), stratégie de distribution multi-langages et repositionnement vis-à-vis du prototype existant

Stage R&D — 4 août 2026

1. Objet du document

Ce document tranche trois questions ouvertes dans la préparation du développement d'Amane : quelle répartition entre Rust et Go pour le cœur du framework, comment le framework sera distribué et consommé par des éditeurs SaaS travaillant dans des langages différents, et pourquoi la réutilisation du prototype existant (application desktop développée par un précédent stagiaire) est limitée au vu des écarts d'architecture.

2. Répartition Rust / Go — quoi, et pourquoi

Le principe retenu est une séparation par responsabilité plutôt qu'un choix unique de langage : Rust là où la sûreté mémoire est non négociable, Go là où l'écosystème de l'infrastructure distribuée est le plus mature.

2.1 Rust — le cœur cryptographique et le journal

Composant	Justification
Double clé AK / DEK, KEK, Shamir SSS	Manipulation directe de clés en mémoire. L'absence de garbage collector évite qu'une clé reste traînée en RAM après usage (ZeroizeOnDrop).
Journal d'écriture (CBOR, append-only)	Colle directement au chiffrement des blobs ; même langage que le cœur crypto pour éviter une frontière de sérialisation supplémentaire.
Bindings libsodium	Écosystème Rust mature et éprouvé pour la cryptographie appliquée (crates sodiumoxide / libsodium-sys).

2.2 Go — consensus, réseau, orchestration

Composant	Justification
Intégration etcd / Patroni	etcd et son client officiel sont écrits en Go ; l'intégration native évite une couche de FFI ou de bindings tiers.
Réseau maillé WireGuard, découverte réseau	Go est le langage standard de l'infrastructure distribuée moderne (Kubernetes, Docker, etcd) — outillage et bibliothèques réseau les plus riches.
Serveur gRPC exposant le framework	Écosystème gRPC/Protobuf natif et performant en Go, adapté à un service qui expose une API vers l'extérieur.

2.3 La frontière entre les deux

La règle de partage est simple à retenir : Rust gère tout ce qui touche aux clés et au contenu chiffré (rien ne doit fuiter en clair côté Go) ; Go gère la coordination entre machines et l'exposition réseau du framework. Le point de passage entre les deux composants se fait via des appels FFI (bindings Rust exposés à Go) ou via un protocole interne, mais jamais via une réimplémentation de la cryptographie côté Go.

3. Distribution du framework en plusieurs langages

Les éditeurs SaaS qui consommeront Amane travaillent dans des stacks hétérogènes (Django/Python, Node/JavaScript, Java, etc.). L'objectif est qu'aucun d'eux n'ait besoin de connaître Rust ou Go pour intégrer le framework.

3.1 Le principe : contrat d'interface unique, clients générés

Le framework expose ses services via des définitions Protocol Buffers (.proto), qui décrivent une seule fois les opérations disponibles (enrôlement, écriture, lecture, révocation, etc.). Le compilateur gRPC génère ensuite automatiquement, à partir de ce même contrat, un SDK client natif pour chaque langage cible : Python, JavaScript/TypeScript, Java, Go, C#, entre autres.

- Un éditeur en Django consomme un SDK Python généré — il l'installe comme une dépendance classique (pip).
- Un éditeur en Node.js consomme le SDK JavaScript équivalent, généré depuis le même .proto.
- Le contrat ne change pas d'un langage à l'autre : une évolution du .proto régénère tous les SDK en une passe.

3.2 Pourquoi cette approche plutôt qu'une librairie par langage

Écrire et maintenir une implémentation manuelle du client dans chaque langage cible multiplierait les risques de divergence (un comportement différent en Python et en JavaScript pour la même opération) et le coût de maintenance à chaque évolution du framework. Générer les SDK depuis un contrat unique garantit que tous les clients respectent exactement la même sémantique, avec un seul endroit à faire évoluer.

3.3 Ce que ça implique pour l'architecture

Cette approche découple strictement le langage interne du framework (Rust + Go) du langage de l'éditeur SaaS qui le consomme. Le served gRPC (en Go) est le seul point de contact réseau du framework ; le cœur Rust reste invisible depuis l'extérieur, accessible uniquement via ce service. C'est cette couche gRPC qui rend le framework véritablement distribuable, alors que le prototype actuel (application desktop) ne l'est pas.

4. Réutilisation du prototype existant — pourquoi elle est limitée

Le prototype développé par le précédent stagiaire est une application desktop (Windows/Linux) autonome, pas un framework distribuable. Le document de stack technique associé à ce prototype pose des décisions structurantes qui sont, pour plusieurs d'entre elles, incompatibles avec l'architecture cible.

Dimension	Prototype existant	Architecture cible	Compatible ?
Unité de distribution	Binaire natif par OS, un cœur Rust unique installé sur chaque poste client	Service distribué exposé via gRPC, consommé par des éditeurs SaaS tiers	Non
Consensus / failover	Promotion de standby PostgreSQL, sans Raft séparé	etcd (Raft) + Patroni, quorum explicite	Non
Conteneurisation	Explicitement exclue (choix assumé et justifié dans le document existant)	Déploiement conteneurisé pressenti pour l'orchestration Go / etcd	Non
Cœur cryptographique (Rust, libsodium, XChaCha20-Poly1305, X25519, Argon2id)	Implémenté et testé sur le spike	Identique dans son principe	Oui
Modèle de clés (DEK simple, sealed box, enrôlement par QR)	DEK unique par entreprise	Double clé AK / DEK + CRL + KEK	Partiellement

Le constat qui ressort : les fondations cryptographiques (chiffrement, dérivation de clé, scellement) sont directement réutilisables, car elles ne dépendent pas du mode de distribution. En revanche, tout ce qui touche à la façon dont le système est packagé, déployé et rendu disponible à des tiers (binaire natif isolé vs. service réseau multi-tenant) doit

être repensé — ce ne sont pas des ajustements mineurs mais deux architectures de distribution différentes, conçues pour des cas d'usage différents (un poste PME isolé vs. un service consommé par plusieurs éditeurs SaaS).

Conclusion pratique : partir du prototype pour la logique crypto de bas niveau fait gagner du temps et évite de réintroduire des bugs déjà corrigés. Partir du prototype pour l'architecture de déploiement, de consensus ou de distribution ferait perdre du temps, car ces choix devront être défaites avant de pouvoir construire l'architecture cible.

5. Répartition des missions pour 3 personnes

La séparation Rust / Go définie en section 2, associée aux 3 missions du cadrage initial (Cyber, Dev, System Design), donne une répartition naturelle du travail entre 3 personnes. Chacune porte un langage et un périmètre de responsabilité clairs, avec des points de contact explicites entre elles.

Personne / Mission	Langage principal	Périmètre
Mission A — Cybersécurité & Crypto souveraineté	Rust	Double clé AK/DEK, hiérarchie KEK, gestion CRL, Shamir SSS (N=3/K=2), audit d'herméticité (absence de fuite / backdoor).
Mission B — Développement & Intégration logicielle	Rust (cœur) + client du SDK généré	Invariants métier (numérotation, stock), lecture hors-ligne, vérification de licence, appairage QR, consommation du framework via le SDK gRPC généré pour valider le contrat côté client.
Mission C — System Design & Résilience	Go	Intégration etcd + Patroni, réseau maillé WireGuard, réplication CRDT multi-site, serveur gRPC exposant le framework, tenue du document d'architecture vivant.

5.1 Les 3 contrats d'interface entre missions

Ces contrats, hérités du cadrage initial, restent la colonne vertébrale de la coordination entre les 3 personnes — ils définissent précisément ce que chacune doit garantir à l'autre, indépendamment du langage utilisé de chaque côté.

Interface	Missions	Contrat
Interface 1	A ↔ C	Enrôlement & membership : posséder une clé = appartenir au cluster. Le dé-enrôlement (A) déclenche le recalcul du quorum (C).
Interface 2	A ↔ B	Frontière de chiffrement : tout ce qui sort du client vers le relais est chiffré. Aucun clair observable, y compris par le SDK généré.
Interface 3	B ↔ C	Chemin d'écriture : opération (B) → validation (B) → journal → réplication synchrone (C) → confirmation (B), via le service gRPC exposé par C.

5.2 Pourquoi ce découpage plutôt qu'un découpage par fonctionnalité

Découper par langage et par responsabilité technique (plutôt que par fonctionnalité métier, ex. "personne 1 fait la facturation") limite les zones de recouvrement de code : une seule personne touche le cœur Rust crypto, une seule personne touche le serveur Go. Cela réduit le risque de conflits de merge sur du code sensible (clés, consensus) et rend les revues de code plus ciblées, chaque personne étant l'experte de son périmètre lors des Pull Requests qui touchent son interface.

6. Synthèse

- Rust : cœur cryptographique, journal d'écriture — sûreté mémoire, aucune primitive réimplémentée à la main.
- Go : consensus (etcd/Patroni), réseau (WireGuard), serveur gRPC exposant le framework — écosystème natif de l'infrastructure distribuée.

- Distribution : un contrat Protobuf unique, des SDK générés automatiquement dans le langage de chaque éditeur SaaS.
- Prototype existant : à réutiliser pour la logique cryptographique de bas niveau, à ne pas suivre pour l'architecture de déploiement et de distribution, qui répond à un besoin structurellement différent.